Implementation Roadmap

Adaptive Multipath Transport Scheduling in Heterogeneous Networks

A supervised-learning MPTCP scheduler built on the existing ns-3-mptcp code base: audit of the current repository, required fixes, phased plan, evaluation protocol and risk register.

Project	Adaptive Multipath Transport Scheduling in Heterogeneous Networks
Department	Computer Science and Engineering, National Institute of Technology Karnataka
Team	Saurabh Kumar (241CS151), Shikhar (241CS153), Vikash Patel (241CS163)
Inputs analysed	CN_project.pdf (proposal); mptcp_scheduler.zip (repository incl. ns-3-mptcp tree, classical_schedulers/, experiments/, results/)
Repository state	9 commits; working tree has uncommitted edits; adaptive_supervised_scheduler/, dataset/ and ml/ are empty scaffolds

Contents

1. Executive summary . . . 3

2. The proposal as testable requirements . . . 4

3. Audit of the current repository . . . 5

3.1 Layout and what each part does . . . 5

3.2 Data flow of one experiment run . . . 5

3.3 The scheduler interface the learned scheduler must implement . . . 6

3.4 Gap analysis against the objectives . . . 7

4. Defects found in the current baseline . . . 8

5. Roadmap overview . . . 11

6. Detailed phase plan . . . 12

7. Key design decisions . . . 20

7.1 Feature definitions . . . 20

7.2 How to label scheduling decisions . . . 21

7.3 Model class and integration approach . . . 21

7.4 Metric definitions . . . 21

7.5 Evaluation and generalization protocol . . . 22

8. Target repository layout . . . 23

9. Team split and schedule . . . 24

9.1 Suggested division of work . . . 24

9.2 Timeline . . . 24

10. Risk register . . . 26

Appendix A. Reference snippets . . . 27

A.1 Shared base class with lazy telemetry registration (Phase 0, task 0.1) . . . 27

A.2 Events file and planned command lines . . . 27

A.3 Decision-log schema (Phase 2) . . . 27

A.4 Generated model header (Phase 3, task 3.6) . . . 28

A.5 Decision procedure of the learned scheduler (Phase 4, task 4.3) . . . 29

Appendix B. Baseline results as recorded (provisional) . . . 30

Appendix C. References . . . 31

1. Executive summary

The proposal commits the team to three things: (i) build a lightweight supervised-learning MPTCP scheduler that uses RTT, bandwidth, loss rate, congestion window and in-flight data; (ii) compare it against minRTT, BLEST and ECF on goodput, throughput, head-of-line (HoL) blocking, retransmissions and flow completion time (FCT); and (iii) test how well it generalizes to unseen and dynamically changing network conditions.

The repository already contains a working ns-3 simulation harness: a modular multi-path topology builder, link profiles for Ethernet, Wi-Fi, LTE and 5G, five schedulers (single-path TCP, minRTT, RoundRobin, BLEST, ECF), automated benchmark drivers, CSV output and a plotting script. That is a solid start on the *evaluation* half of the project. None of the *learning* half exists yet.

> **Headline finding: fix the baseline before building the learned scheduler**
>
> An audit of the code and of the existing results shows that the classical baselines are not yet trustworthy. The per-subflow telemetry that every scheduler depends on is never populated (`TelemetryTracker::RegisterSubflow()` has no callers), so BLEST, ECF and minRTT run on constant default values and, in the recorded results, BLEST and minRTT are bit-identical. Single-path TCP also outperforms every MPTCP scheduler in every scenario, and some throughput numbers exceed the physical link capacity. A learned scheduler trained or evaluated on this foundation would produce conclusions that cannot be defended.

This document therefore proposes an eight-phase roadmap. Phase 0 repairs the telemetry and the metrics so that the baselines mean what the literature says they mean. Phases 1 and 2 build the scenario engine, the missing metrics and the training dataset. Phases 3 and 4 train a compact model, export it to dependency-free C++ and implement the scheduler. Phases 5 and 6 carry out the comparison (objective ii) and the generalization study (objective iii). Phase 7 is the write-up. The total effort is about nine weeks for a team of three and can be compressed by overlapping Phases 2 and 4.

What this document contains

- **Section 2** restates the proposal as testable requirements.
- **Section 3** is a file-level audit of the repository: architecture, data flow, current results and a gap analysis against each objective.
- **Section 4** lists seven concrete defects with evidence, impact and fix.
- **Sections 5 and 6** give the phase overview and a detailed task breakdown for each phase, with file names, interfaces, deliverables and exit criteria.
- **Section 7** records the key design decisions (labeling, features, model, integration, evaluation protocol, metric definitions).
- **Sections 8 to 10** cover the target repository layout, team split and schedule, and the risk register.
- **Appendices** hold reference snippets, the dataset schema, and the existing benchmark results.

2. The proposal as testable requirements

The proposal (CN_project.pdf) argues that MPTCP lets a device use several interfaces at once, and that the scheduler, which decides which subflow carries each segment, is the component that determines whether that potential is realized. The default minRTT scheduler under-utilizes heterogeneous paths. BLEST and ECF add hand-designed heuristics. Reinforcement-learning schedulers such as ReLeS exist, and supervised approaches have produced datasets, but it remains an open practical question whether a supervised scheduler trained under some conditions generalizes to unseen, changing ones. That question is the heart of the project.

Objectives mapped to measurable acceptance tests

Obj.	Proposal wording (condensed)	Acceptance test used in this roadmap
(i)	Build a supervised-learning MPTCP scheduler that decides how to split traffic across paths from RTT, bandwidth, loss rate, cwnd and in-flight data.	A scheduler class selectable via `--protocol=Supervised`; takes exactly these seven per-subflow features at every scheduling decision; runs with no runtime ML library; adds negligible decision latency.
(ii)	Compare with minRTT, BLEST, ECF on goodput, throughput, HoL blocking, retransmissions, FCT.	All five metrics collected for every scheduler on every scenario, over at least 10 seeds, reported with 95% confidence intervals and paired significance tests.
(iii)	Investigate generalization on conditions that differ from training, including dynamic changes in bandwidth, latency and packet loss.	A pre-declared train/test protocol with held-out topologies, held-out parameter ranges and unseen dynamic patterns; the generalization gap is reported for the learned scheduler and for every baseline.

Implicit requirements worth stating explicitly

- **Fairness of comparison.** If the baselines are given privileged information (for example the true link rate read from the simulator), the learned scheduler must get the same information, or none of them should.
- **Lightweight.** The word appears in the problem statement. The model must be small enough to run per segment inside the scheduler without measurable overhead, which argues for trees, forests or small linear models over deep networks.
- **Generalization is the research question.** A scheduler that beats the baselines only on the scenarios it was trained on would answer the wrong question. The evaluation design in Section 7.5 is built around this.

3. Audit of the current repository

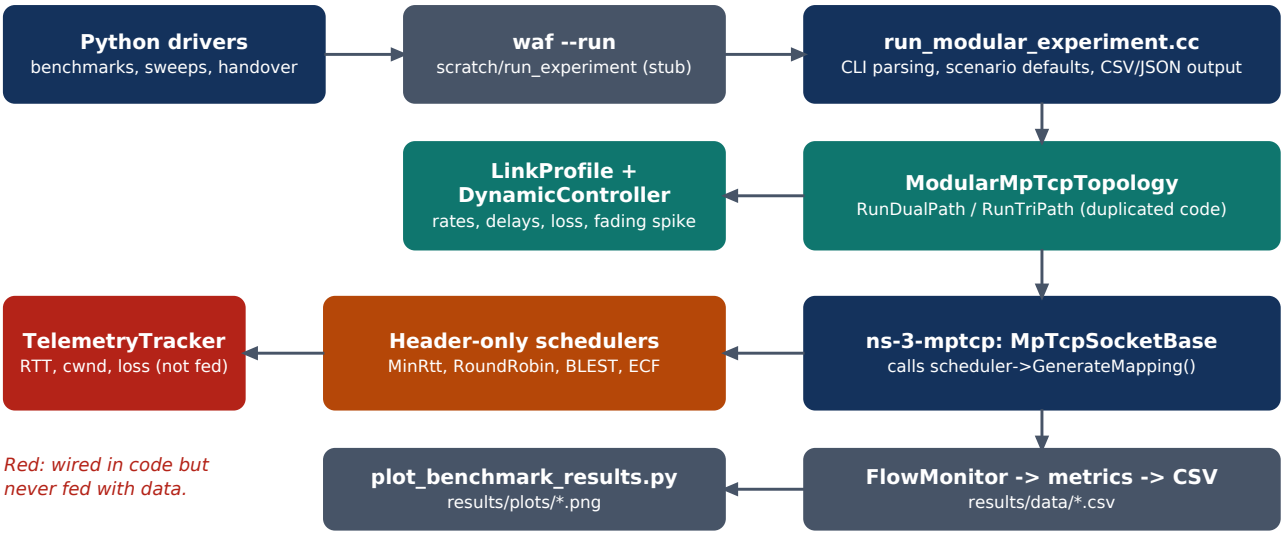
3.1 Layout and what each part does

The zip contains a complete ns-3-mptcp source tree (about 5,600 files, roughly 1.5 GB with build products) plus the team's own directories. Only the team's directories are under version control: `ns-3-mptcp/` is listed in `.gitignore`.

Path	Contents	State
classical_schedulers/	Header-only `MpTcpSchedulerMinRtt`, `RoundRobinCustom`, `BlestCustom`, `EcfCustom` and `telemetry_tracker.h`. Also older `.cc` files and a copy of the base scheduler.	Used (headers). The `.cc` files are not compiled and use older class names; they look stale.
experiments/	`run_modular_experiment.cc` (CLI entry point), `topologies/{link_profile,dynamic_controller,modular_mptcp_topology}.h`, three Python drivers, `plot_benchmark_results.py` (22 KB).	Working. Metrics and dynamics are limited (Section 3.4).
results/	Four CSVs in `data/` and 14 PNG plots in `plots/`.	Generated from the current (flawed) baseline; must be regenerated.
adaptive_supervised_scheduler/	Empty.	To build (Phase 4).
dataset/raw, dataset/processed	Empty.	To build (Phase 2).
ml/models	Empty.	To build (Phase 3).
ns-3-mptcp/scratch/run_experiment.cc	One line: `#include "../../experiments/run_modular_experiment.cc"`.	Glue so that `waf --run scratch/run_experiment` builds the team's code.
ns-3-mptcp/build/ns3/mptcp-scheduler-supervised.h	A header-only declaration of a supervised scheduler with a `SubflowTelemetryFeature` struct matching the PDF.	Stale build artefact (no implementation, read-only). Useful as a design reference.

3.2 Data flow of one experiment run

A Python driver launches the compiled `run_experiment` binary through waf with a topology, scheduler name, duration and optional overrides. The runner builds the topology (point-to-point links with BDP-sized drop-tail queues and receive-side error models), sets `ns3::MpTcpSocketBase::Scheduler` to the chosen scheduler's TypeId, runs a BulkSend application for the configured duration plus a 2-second drain, and then aggregates FlowMonitor statistics into an `ExperimentMetrics` struct that is printed as text or JSON and appended to a CSV. The plotting script reads the CSVs.



This architecture is a good fit for the project. Because schedulers are header-only classes included directly into the experiment program, a new scheduler needs no change to the ns-3 library build, and the `Scheduler` attribute makes it selectable at run time. That is exactly the integration point the learned scheduler will use.

3.3 The scheduler interface the learned scheduler must implement

Every scheduler derives from `MpTcpScheduler` and implements two methods: `SetMeta(Ptr<MpTcpSocketBase>)` and `bool GenerateMapping(int& activeSubflowArrayId, SequenceNumber64& dsn, uint16_t& length)`. The meta socket calls `GenerateMapping` whenever it can send. The scheduler picks a subflow index, the data-sequence number (taken from the meta socket's next transmit sequence) and a length, or returns false to wait. The existing schedulers read subflow state through public accessors: `AvailableWindow()`, `BytesInFlight()`, `GetSegSize()`, and ns-3 trace sources RTT and `CongestionWindow`. All seven features named in the proposal can be derived from these without modifying ns-3 internals, except retransmission counts and loss, which need either a trace or a small patch (Section 6, Phase 1).

3.4 Gap analysis against the objectives

Requirement	Present	Missing
Heterogeneous topologies	WiFi+LTE (12M/15ms + 6M/45ms), Ethernet+WiFi (25M/5ms + 15M/20ms), tri-path (30M/5ms + 15M/20ms + 10M/50ms); per-path overrides on the CLI.	A single generic N-path builder (dual and tri-path code is duplicated, about 150 lines each); randomized scenario generation.
Baseline schedulers	minRTT, RoundRobin, BLEST, ECF, single-path TCP.	Working telemetry (Section 4, defect 1); a verified-correct BLEST and ECF; a default Linux-style minRTT reference.
Goodput / throughput	Computed from FlowMonitor over a fixed 7.5 s divisor.	Common measurement window; application-level goodput from the sink; correct handling of the drain period.
HoL blocking	Nothing.	Receiver-side measurement of reordering delay and out-of-order buffer occupancy.
Retransmissions	Nothing (loss is derived from FlowMonitor lostPackets).	Per-subflow retransmission counter (hook at `MpTcpSubflow::DoRetransmit`).
Flow completion time	Nothing; BulkSend runs with `MaxBytes=0` (unbounded).	Fixed-size flows and a completion-time probe at the sink.
Dynamic conditions	One hard-coded 8% loss spike on path 0 between 4 s and 7 s (dual-path only). `ApplyRateChange` exists but is never called.	Time-varying bandwidth, delay and loss; scripted and random scenarios; path-quality swaps.
Statistical rigor	One run per configuration, 6-10 s long.	Seeds, repeats, confidence intervals, longer runs.
ML pipeline	Nothing.	Feature logging, labeling, training, model export, scheduler class.

4. Defects found in the current baseline

The findings below come from reading the code, from the committed result files, and from two short test runs (FastestRTT and BLEST, WiFi_Cellular, 4 s) of the prebuilt binary. Where a conclusion is an inference rather than something directly observed, the text says so. Each defect lists the evidence, why it matters for the project, and the fix, which maps to a task in Phase 0 or Phase 1.

Evidence table: recorded throughput in Mbps (results/data/multitopology_benchmark.csv)

Scenario (capacity)	Standard-TCP	FastestRTT	RoundRobin	BLEST	ECF
WiFi + LTE (18 Mbps)	4.95	4.59	3.64	4.59	3.62
Ethernet + WiFi (40 Mbps)	29.28	24.27	12.36	24.27	12.53
Tri-path (55 Mbps)	37.34	26.87	4.82	26.87	6.24
WiFi + LTE with 8% fade, 4-7 s	5.24	3.41	3.01	3.41	3.03

Two patterns are visible at once: FastestRTT and BLEST are identical to every printed digit in every row, and the first column (single-path TCP, which uses only path 0) is the highest or second-highest in all rows. The tri-path single-path figure, 37.34 Mbps, is higher than the 30 Mbps Ethernet link it runs over.

Defect 1 (critical): scheduler telemetry is never populated

- **What the code does.** All four custom schedulers obtain RTT and bandwidth from `TelemetryTracker::Get().GetTelemetry(i)`. The tracker only learns RTT and cwnd if `RegisterSubflow(id, subflow, linkRate)` is called, because that method connects the RTT and `CongestionWindow` trace sources and records the link rate.
- **Evidence.** A search of `classical_schedulers/` and `experiments/` finds no caller of `RegisterSubflow` and none of `RecordLoss`. `UpdateFromSocket` refreshes only in-flight bytes, available window and MSS. The tracker therefore keeps its constructor defaults for every subflow: RTT 20 ms, bandwidth 1.25 MB/s, loss rate 0.
- **Consequences.** (a) With equal RTTs, the strict comparison `tel.rtt < lowestRtt` always picks subflow 0, so minRTT degenerates to "use the primary subflow, spill over when its window is full". (b) BLEST's first test, `slowTel.rtt <= fastTel.rtt`, is always true, so the blocking logic never fires and BLEST equals minRTT, which is exactly what the CSVs show and what the two test runs reproduced (both gave 4.44381 Mbps with identical per-path byte counts). (c) ECF's completion-time estimate uses constant RTT and rate. (d) The loss-rate feature the proposal lists is always zero.
- **Why it matters.** The learned scheduler is supposed to be compared against BLEST and ECF, and it is supposed to use RTT, bandwidth and loss as features. As things stand, neither the baselines nor the feature pipeline work.
- **Fix.** Register each subflow when it becomes active (lazy registration inside the scheduler, Appendix A.1), feed bandwidth from a measured delivery-rate estimator, and feed loss from a retransmission count (Phase 0, tasks 0.1 to 0.3). A small related bug: `UpdateRtt` uses the value 20 ms as a "not yet set" sentinel for the minimum RTT, which would misbehave for a path whose true RTT is 20 ms.

Defect 2 (critical): BLEST and ECF are not the published algorithms

Even with working telemetry, the implementations differ from the papers cited in the proposal. The BLEST header compares estimated *delivery times* of the slow and fast subflows, which resembles ECF's idea. The BLEST paper (Ferlin et al., IFIP Networking 2016) instead estimates how many bytes the fast subflow could send during one slow-path RTT and blocks the slow subflow if sending on it would exhaust the shared send window before the fast path can free it, with an adaptive correction factor. The header's own comment also expands the acronym incorrectly ("Local Energy-aware Scheduling"). ECF (Lim et al., CoNEXT 2017) uses the number of bytes still to send, the congestion windows of both subflows and a hysteresis margin to decide whether to *wait* for the fast subflow instead of using a slow one. The ECF header actually used by the experiments never waits: it picks the subflow with the smallest `RTT + inflight/rate` among those with a free window. With the constant telemetry of defect 1 this reduces to balancing by in-flight bytes, which is consistent with ECF's RoundRobin-like results (3.62 vs 3.64 Mbps on WiFi + LTE). Reviewers who know these papers will check this. Re-implement both from the published equations (Phase 0, task 0.3) and state any simplification explicitly in the report.

Defect 3 (high): single-path TCP beats every MPTCP scheduler

In all four scenarios, a single TCP connection on path 0 matches or beats every MPTCP scheduler. In the Ethernet + WiFi case the single path reaches 29.3 Mbps while the best MPTCP scheduler reaches 24.3 Mbps, despite 15 Mbps of additional Wi-Fi capacity. A correct multipath transport should not lose to its own primary path in a lossless, well-buffered topology. Defect 1 explains part of this, since the schedulers are effectively primary-first with no intelligence. Other plausible contributors are head-of-line blocking from spill-over onto the slower path, subflow congestion control interacting with the 0.5 to 1.5% random loss, and the 4 MB buffers. These are hypotheses to be tested with the sanity suite in task 0.8, not conclusions. Until the baseline behaves sensibly in simple cases, no comparison involving it is meaningful.

Defect 4 (high): throughput and efficiency are over-reported

The runner divides received bytes by `duration - 0.5 s` (7.5 s in the default benchmark), but the sink application keeps running for a further 2 s drain period, and the bytes counted by FlowMonitor include everything received until then, including the contents of the 4 MB send buffer draining after the sender stops. This most likely explains why single-path TCP appears to carry 37.34 Mbps over a 30 Mbps link: 30 Mbps over about 9.5 s divided by 7.5 s gives about 38 Mbps. The inflation differs between schedulers because it depends on how much data is queued when the sender stops, so it also biases the comparison. Aggregation efficiency inherits the error. Fix: measure goodput at the sink over a single fixed window that ends when the sender stops (task 0.4).

Defect 5 (high): several reported metrics are not measurements

- **Session goodput** is computed as `throughput x (1 - lost/sent)`. That is a discounted throughput, not goodput. True goodput counts only new application bytes delivered in order to the receiving application, which excludes duplicate and retransmitted data.
- **Jitter** is set to `0.15 x mean delay`. It is a constant fraction of another metric.
- **Packet loss** reads 0.00% in most rows even though the links are configured with 0.5 to 1.5% random loss (it is only non-zero for Standard-TCP in the tri-path run). The loss is applied by a receive-side error model, so it is either being hidden by TCP retransmission or not classified as loss by FlowMonitor. Either way the column does not reflect what happens in the network.
- **Absent metrics.** HoL blocking, retransmissions and FCT, three of the five metrics named in objective (ii), are not collected.

Defect 6 (medium): the disparity sweep axis is inconsistent

In `run_modular_experiment.cc` the override is applied only when `disparity > 1.0`. The label "1x" therefore uses the default 15 ms / 45 ms pair, a 3x delay ratio, while "2x" sets path 1 to 30 ms (a 2x ratio), "4x" to 60 ms, and so on. The ratio goes 3, 2, 4, 8, 16, so the axis is not monotonic, which is a likely reason the goodput curve peaks at 2x (6.36 Mbps vs 4.51 Mbps at "1x"). Define disparity as the ratio of path-1 to path-0 one-way delay for every value, including 1.0 (task 0.6).

Defect 7 (medium): statistical and engineering hygiene

- **One run per cell, 6 to 10 s long.** No seed control, so no variance estimate, and short runs are dominated by TCP slow start. The comparisons in the current plots cannot support significance claims.
- **Information asymmetry.** The older `.cc` BLEST/ECF read the physical link `DataRate` from the net device, an oracle value no real host has. The header versions use the tracker's rate. The project needs one explicit policy for all schedulers including the learned one (recommended: measured delivery rate for all).
- **Version control gaps.** The simulator tree is untracked, so any ns-3 patch (needed for retransmission counters and HoL measurement) would be invisible to collaborators. The working tree also has many uncommitted modifications and deletions, including the README, and the last commit does not contain the current benchmark scripts.
- **Dead and duplicate code.** `classical_schedulers/*.cc` are not compiled; `mptcp-scheduler.h` and `mptcp-scheduler-fastest-rtt.*` duplicate the files in ns-3; dual-path and tri-path topology code is copy-pasted; the runner ends with `_exit(0)`, which skips normal teardown.

> **Practical consequence**
>
> Treat every number in `results/` as provisional. Keep the plots as a record of the starting point, but regenerate them after Phase 0 and do not cite them in the report.

5. Roadmap overview

The plan is organized as eight phases. Each phase has a defined deliverable and an exit check that must pass before the next phase relies on it. The ordering is deliberate: measurement before modelling, data before training, and the generalization study last, because it needs every earlier component to be correct.

Phase	Name	Effort	Main deliverables	Depends on
0	Repair the baseline	~1 wk	Working telemetry; paper-faithful BLEST and ECF; corrected metrics; seeds; sanity suite; clean repo	-
1	Scenario engine and metrics	~1 wk	N-path builder; event-driven dynamics; HoL, retransmission, FCT metrics; workloads; ns-3 patches	0
2	Dataset generation	~1.5 wk	Decision logger; exploration policies; scenario sampler; labeled dataset with leak-free splits	0, 1
3	Model training and export	~1 wk	Model comparison; selected lightweight model; generated C++ header; equivalence test	2
4	Learned scheduler	~2 wk (overlaps 3)	`MpTcpSchedulerSupervised`; fallback guard; registration; driver updates	0, 1; 3 for the final model
5	Comparative evaluation (obj. ii)	~1.5 wk	Full matrix, CIs, significance tests, plots, oracle ceiling	3, 4
6	Generalization study (obj. iii)	~2 wk	Held-out topologies, ranges, dynamics, workloads; ablations; gap analysis	5
7	Write-up and packaging	~1.5 wk	Report, figures, reproducibility package, threats to validity	5, 6 (starts earlier)

Guiding principles

- **One feature function.** The code that turns subflow state into the model's input must be shared by the data logger and the scheduler. Divergence between them (train/serve skew) is the most common silent failure in learned systems.
- **Same information for every scheduler.** All schedulers read the same telemetry source, so any performance difference comes from the decision rule.
- **Split by scenario, never by row.** Consecutive decisions in a run are strongly correlated. Random row-wise splits would leak and overstate accuracy; grouped splits by scenario and by scenario family are required.
- **Offline accuracy is not the goal.** Models are selected by simulated goodput, FCT and HoL delay, because a model with a good offline error can still schedule badly.
- **Pre-declare the generalization protocol.** Write the train/test scenario families down before looking at test results, so that the study cannot be accused of tuning to the test set.
- **Negative results are acceptable.** The proposal asks whether a supervised scheduler generalizes. If it does not, with a clear characterization of where it breaks, that is a valid finding.

6. Detailed phase plan

Each phase below lists its goal, a task list with file-level detail, the deliverables, the exit criteria and the main risks. Task numbers (for example 0.3) are referenced from other sections.

PHASE 0	Repair the baseline	~1 week Owner: simulator/metrics

Goal. Make the classical schedulers and the measurements trustworthy, so that every later comparison rests on correct numbers. Nothing in Phases 2 to 6 should start until the exit criteria hold.

Tasks

1. **0.1 Wire the telemetry (fixes defect 1).** Add `classical_schedulers/telemetry_scheduler.h`, a base class derived from `MpTcpScheduler` that every scheduler inherits. Its `EnsureRegistered()` iterates over `GetNActiveSubflows()` and calls `TelemetryTracker::RegisterSubflow` once for each new subflow (subflows appear after the handshake, so registration must be lazy, on the first `GenerateMapping` call). A `RefreshAll()` method updates in-flight bytes, available window and MSS for every subflow at the start of each decision. See Appendix A.1.
2. **0.2 Make the telemetry correct.** (a) Replace the 20 ms minimum-RTT sentinel by an explicit "first sample" flag. (b) Replace the configured link rate by a *measured* delivery-rate estimator: connect the `HighestRxAck` trace source of each subflow, accumulate newly acknowledged bytes and smooth with an EWMA over a 100 ms window. Use this estimate in all schedulers. Keep a switch `--oracleBandwidth=1` only as an experimental ablation. (c) Derive an interim loss signal from the `CongState` trace (count of transitions into recovery or loss states per subflow) until the exact retransmission counter of Phase 1 exists. (d) Remove the singleton's reliance on global state between runs by resetting it in each topology run (already done for `Reset()`; add a test).
3. **0.3 Re-implement BLEST and ECF from the papers.** BLEST: for the slow subflow s and fast subflow f, estimate $X = MSS_f \times (cwnd_f + (rtt_s/rtt_f - 1)/2) \times rtt_s/rtt_f$ (bytes the fast path could send while one slow-path segment is in flight) and skip the slow subflow if lambda x X exceeds the free space in the connection-level send window after accounting for the slow subflow's in-flight data, adapting lambda up when blocking is detected and down otherwise. ECF: compare the time to finish the remaining queued data on the fast subflow alone against the time to use the slow subflow, using remaining bytes, both congestion windows and a hysteresis flag, and wait for the fast subflow when it is faster overall. Verify constants and conditions against Ferlin et al. (2016) and Lim et al. (2017) while coding, and document any simplification. Rename classes to drop the "Custom" suffix once the stale `.cc` files are removed.
4. **0.4 Fix the metrics (defects 4 and 5).** Measure goodput at the sink as new in-order application bytes divided by a single fixed window $[t_{start}, t_{stop}]$, where t_{stop} is when the sender stops. Use the `PacketSink` Rx trace to timestamp cumulative bytes, so the drain period does not count. Report wire throughput separately (all subflow bytes including headers and retransmissions over the same window). Delete the constant-fraction jitter and compute jitter as the standard deviation of inter-delivery times at the sink. Report packet loss only from counters whose meaning is documented (Phase 1 supplies them).
5. **0.5 Seeds and repeats (defect 7).** Add `--seed` and `--run` arguments that call `RngSeedManager::SetSeed` and `SetRun` before topology creation, make the Python drivers loop over at least 10 runs, and extend the CSV schema with `seed,run`. Lengthen runs to 30 s for steady-state metrics and compute the metrics over the last 20 s so slow start does not dominate.
6. **0.6 Fix the disparity axis (defect 6).** Always set path-1 delay to path-0 delay times the disparity factor, including 1.0, and record the realized ratio in the CSV.
7. **0.7 Repository hygiene.** Commit the current state first (so history shows the starting point), then: write a README with build and run steps; add a `patches/` directory for ns-3 changes with an

`apply_patches.sh`; delete or archive the stale `classical_schedulers/*.cc`, the duplicate base-scheduler header and the old fastest-RTT files; make the Python drivers share one `common.py` (paths, protocol list, CSV helper); add a `scripts/run_all.sh` that builds and runs everything; pin the Python dependencies in a `requirements.txt`.

8. **0.8 Sanity suite.** Add `experiments/sanity/` with tests whose expected outcomes are written down *before* running them: (a) a single link at C Mbps: single-path TCP must stay below C; (b) two identical lossless paths with large buffers: MPTCP should reach clearly more than a single path (record the actual ratio and investigate if it is below about 1.6x); (c) two paths with a large delay gap: BLEST and ECF should show lower reordering delay than minRTT; (d) a path with total outage: MPTCP should keep delivering over the other path. Failures here point at defects in the baseline, and fixing them is part of this phase.

Deliverables

- Corrected schedulers and tracker; corrected metrics; sanity-suite report; a clean, committed repository; regenerated baseline results with confidence intervals.

> **Exit criteria**
> BLEST, ECF and minRTT produce different and explainable results; the sanity suite passes or each failure has a documented cause; no reported throughput exceeds physical capacity; results over 10 seeds show variance estimates; the repository builds from a fresh clone using the README.

Risks. Fixing telemetry may expose further bugs in the MPTCP stack itself (the ns-3-mptcp fork is old). Budget extra time for task 0.8(b); if MPTCP still underperforms single-path TCP, investigate the subflow congestion control and buffer settings before blaming the schedulers.

PHASE 1 — Scenario engine and metrics

~1 week
Owner: simulator/metrics

Goal. Build everything objectives (ii) and (iii) need from the simulator: arbitrary multi-path topologies, time-varying conditions, realistic workloads, and the three missing metrics.

Tasks

1. **1.1 Generic N-path topology.** Replace `RunDualPath` and `RunTriPath` by one `Run(std::vector<LinkProfile>)` that creates one router per path, assigns addresses by index and installs static routes in a loop. Keep the existing named topologies as presets that return profile vectors. This removes about 300 lines of duplication and makes 2, 3 and 4-path experiments a parameter change.
2. **1.2 Event-driven dynamics.** Extend `DynamicController` with `ApplyDelayChange` (set the `Delay` attribute of the `PointToPointChannel`, which exists and is settable at run time), keep `ApplyRateChange` and `ApplyErrorRate`, and add a loader for an events file `--events=events.csv` with columns `time_s,path,rate_mbps,delay_ms,loss` (blank means unchanged). A plain CSV avoids needing a JSON library in C++. A Python generator (task 2.2) writes these files.
3. **1.3 Dynamic pattern library.** Implement as event generators: bandwidth step-down and step-up; linear ramp; periodic fading (square and sinusoidal); delay spike (bufferbloat); loss burst; path outage and recovery; "quality swap" where the fast path becomes slow and vice versa (the case where heuristics that assume a stable fast path fail); and handover (one path leaves, another joins, via interface down/up).
4. **1.4 Metrics collector.** Add `experiments/metrics/metrics_collector.h` that owns all measurement: sink-side cumulative-byte trace; per-subflow counters; time series in 100 ms bins written to `results/timeseries/*.csv` for the dynamics plots; and the final summary row.

5. **1.5 Retransmission counter.** Patch `MpTcpSubflow::DoRetransmit()` (mptcp-subflow.cc, near line 309) to increment a per-subflow counter and fire a new traced callback. Save as `patches/0001-subflow-retx-trace.diff`. Report retransmitted segments per subflow and as a fraction of segments sent.
6. **1.6 HoL-blocking measurement.** Define it at the receiver as the delay between a segment's *arrival* at the meta socket and its *release in order* to the application, plus the time series of out-of-order buffer occupancy. This needs a hook where the meta receive buffer accepts DSS-mapped data and where it releases in-order data in `mptcp-socket-base.cc` (the exact insertion point has to be located in the source; it was not pinned down in this audit). If a patch proves hard, a documented fallback proxy is: in-order delivery time at the sink minus (send time + the chosen subflow's smoothed one-way delay). Save any patch as `patches/0002-meta-rx-hol-trace.diff`.
7. **1.7 Workloads.** Provide (a) unbounded BulkSend for steady-state throughput; (b) fixed-size transfers (100 KB, 1 MB, 10 MB) via `MaxBytes` with flow completion time measured from the first byte sent to the sink having received all bytes; (c) optionally an on/off source (short bursts with think time) so training and testing workloads can differ in Phase 6.
8. **1.8 Run manifest.** Every run writes a `manifest.json` (git commit, scenario file hash, seed, scheduler, CLI) next to its output so that any plotted number can be traced to its exact configuration.

Deliverables

- Generic topology builder; events-file dynamics and a pattern library; metrics collector with goodput, wire throughput, jitter, retransmissions, HoL delay and FCT; ns-3 patches under version control; run manifests.

> **Exit criteria**
>
> A scripted dynamic scenario (for example bandwidth halves at t=10 s) shows the expected step in the goodput time series; retransmission counts are non-zero on lossy paths and zero on clean ones; HoL delay is near zero for single-path TCP and positive for multipath over mismatched paths; FCT for a 1 MB transfer matches a back-of-envelope estimate within a reasonable margin.

Risks. Point-to-point links are a simplification of Wi-Fi and cellular (no contention, no scheduler variability). Compensate with the dynamics library, and state the limitation in the threats-to-validity section.

PHASE 2 — Dataset generation

~1.5 weeks
Owner: dataset/ML

Goal. Produce a labeled dataset of scheduling decisions that a model can learn from, with splits that support an honest generalization test.

Tasks

1. **2.1 Decision logger.** A header `adaptive_supervised_scheduler/decision_logger.h` used by every scheduler. On each successful `GenerateMapping` it writes one row: time, run id, DSN, length, chosen subflow, number of active subflows, and for each subflow slot the seven features (RTT, minimum RTT, bandwidth estimate, cwnd, bytes in flight, available window, loss rate). Slots are padded to a maximum of 4 paths with a validity mask. Schema in Appendix A.3. Logging is off by default and enabled with `--logDecisions=path`.
2. **2.2 Scenario sampler.** `ml/sample_scenarios.py` draws scenarios by Latin hypercube sampling over per-path rate (2 to 50 Mbps), one-way delay (2 to 100 ms), random loss (0 to 3%), and queue size (0.5 to 3 x BDP), for 2 and 3 paths, with a dynamics pattern chosen from a *training subset* of the Phase 1 library. Patterns reserved for testing are never sampled here. The sampler writes scenario files and a manifest of which scenario belongs to which family, and is seeded for reproducibility.
3. **2.3 Data-collection policies.** Run every sampled scenario under several policies so that the data covers decisions a learner will later want to make: the repaired minRTT, BLEST and ECF schedulers; RoundRobin; and an epsilon-random policy (epsilon values 0.1, 0.3, 1.0) that picks a random eligible subflow. Without exploration the logs contain only the choices a heuristic makes and the model cannot learn what would have happened on the alternatives (Section 7.2).
4. **2.4 Outcome join and labels.** The sender logs when each DSN was sent. At the sink, the cumulative in-order byte trace gives the time at which byte offset k was delivered to the application, so DSN minus the initial DSN gives a stream offset whose delivery time is known without touching ns-3 internals. The primary label is the *per-segment delivery delay* = delivery time minus send time, and, if the Phase 1 hook exists, its HoL component. A secondary label is the oracle choice (Section 7.2). Implement as `ml/build_dataset.py`.
5. **2.5 Splits.** Write `dataset/processed/splits.json` assigning whole scenarios to train, validation and test, plus separate held-out sets for Phase 6: unseen topology (3-path when training used 2-path, or the reverse), unseen parameter ranges (for example delay disparity beyond the training maximum), unseen dynamics patterns, and an unseen workload. Never split by row.
6. **2.6 Data card.** Document the dataset: how it was generated, counts per family, feature definitions and units, known biases, and the commit hash of the simulator that produced it.

Sizing and compute

A 30 Mbps flow sends roughly 2,600 segments per second, so a 20 s run yields on the order of 50,000 rows. A campaign of a few hundred scenarios under six policies is millions of rows, which a laptop handles comfortably as compressed CSV or Parquet. Wall-clock simulation time is the real constraint: measure the time per run first and then parallelize across CPU cores with a process pool, running each simulation as an independent process.

Deliverables

- Decision logger; scenario sampler; raw logs in `dataset/raw/`; labeled data and split manifests in `dataset/processed/`; data card.

Exit criteria

Row counts and label distributions are plausible per family; a spot check of 20 random rows reconciles send time, delivery time and chosen subflow against a packet trace; no scenario appears in more than one split; the exploration policies visibly cover subflow choices that heuristics rarely make.

Risks. (a) Delivery delay depends on other segments in flight, not only on the chosen subflow, which makes the label noisy. Mitigate with scenario-level grouped validation and with features that capture queue state. (b) Exploration hurts the quality of the runs that generate the data, which is acceptable for training data but means those runs must never be used as evaluation results.

PHASE 3 — Model training and export

~1 week
Owner: dataset/ML

Goal. Choose a small, fast model that predicts a good subflow, and turn it into dependency-free C++.

Tasks

1. **3.1 Feature engineering.** Beyond the seven raw features, add scale-free relative features per subflow, for example RTT divided by the minimum RTT across subflows, bandwidth share of the total, window headroom as a fraction of cwnd, and RTT inflation over the path's own minimum (a bufferbloat indicator). Relative features are what allow a model trained on 5 to 50 Mbps paths to apply to paths outside that range, which directly supports objective (iii).
2. **3.2 Model formulation.** Use one *shared per-subflow scoring model*: its input is that subflow's features plus a small summary of the other subflows (fastest RTT, total window, number of paths), and its output is the predicted delivery delay if the next segment goes on that subflow. The scheduler picks the subflow with the lowest prediction. Because the same model scores every subflow, it works for any number of paths without retraining, which allows 2-path to 3-path generalization tests. A classification formulation (one output per subflow plus a "wait" class) is the alternative and is kept as an ablation.
3. **3.3 Candidate models.** Ridge or logistic regression as the linear baseline; a depth-limited decision tree; a random forest and a gradient-boosted tree ensemble with small limits (for example up to 50 trees of depth 6 to 8). Scikit-learn is sufficient. A small MLP can be included as an upper reference but is not preferred, since the proposal asks for a lightweight scheduler.
4. **3.4 Grouped validation.** Use GroupKFold with scenario id as the group, and a separate pass grouped by scenario family to estimate how performance changes under distribution shift. Report offline mean absolute error of predicted delay, agreement with the oracle choice, and decision regret (delay of the model's choice minus the best subflow's delay).
5. **3.5 Size and latency budget.** Set the budget before training: for example generated code below 100 KB and inference below about 5 microseconds per segment, measured by a microbenchmark. Models above the budget are rejected however accurate they are.
6. **3.6 Export to C++.** `ml/export_cpp.py` writes `adaptive_supervised_scheduler/model_generated.h`: feature scaling constants and the trees as flat arrays (feature index, threshold, left, right, leaf value) with a short traversal loop. No dynamic allocation, no external libraries. Linear models export as a weight vector. Appendix A.4 shows the format.
7. **3.7 Equivalence test.** `ml/test_export.py` evaluates the C++ model (compiled to a tiny test binary) and the scikit-learn model on 10,000 random feature vectors and requires the maximum absolute difference to be below 1e-5. This catches threshold rounding and indexing errors.

Deliverables

- Trained candidate models with a comparison table; the selected model saved in `ml/models/`; generated header; equivalence test; model card (training data hash, hyperparameters, size, latency).

> **Exit criteria**
> Selected model meets the size and latency budget, beats a constant-prediction and a minRTT-imitation baseline on grouped validation, and the C++ export is numerically equivalent to the Python model.

Note. Offline accuracy only shortlists models. The final choice is made in Phase 5 by simulated performance on a validation scenario set, not on the test set.

PHASE 4 — The learned scheduler

~2 weeks (overlaps Phase 3)
Owner: scheduler/eval

Goal. Implement the scheduler as a drop-in class with the same interface and the same telemetry as the baselines. Work can start during Phase 2 with a hand-written stand-in model (for example a linear scoring rule) so that the plumbing is tested before real models exist.

Tasks

1. **4.1 Class and interface.** Create `adaptive_supervised_scheduler/mptcp-scheduler-supervised.h` defining `MpTcpSchedulerSupervised : TelemetryScheduler` (header-only, same pattern as the other schedulers). Reuse the feature struct and the method names from the stale `build/ns3/mptcp-scheduler-supervised.h` (`ExtractTelemetry, SelectSubflow, PredictOptimalSubflowTree`), adapted to the generated model header.
2. **4.2 One feature function.** Implement `FeatureExtractor::Build(telemetry, subflowIndex, allSubflows)` in a single header included by both the decision logger and the scheduler. Add a golden-file test that feeds a fixed telemetry snapshot through the extractor and compares the output vector with a vector computed by the Python training code, so train/serve skew is caught immediately.
3. **4.3 Decision rule.** At each `GenerateMapping`: refresh telemetry; collect eligible subflows (available window at least one MSS); compute predicted delay for each; choose the minimum. Optional "wait" rule: if the best eligible subflow's prediction is worse than the predicted wait-for-the-fast-path time by a margin, return false (this is the behaviour BLEST and ECF use and is where learned schedulers often gain). Return the DSN from the meta socket's next transmit sequence and a length bounded by MSS, subflow window and meta window, exactly as the baselines do.
4. **4.4 Safety guard.** If any feature is invalid (for example no RTT sample yet) or the model output is not finite, fall back to the repaired minRTT rule for that decision and count the fallback. Report the fallback rate; a high rate means the scheduler is not really running the model. During the first RTTs, probe each subflow once so that every path has an RTT sample.
5. **4.5 Registration and drivers.** Add `NS_OBJECT_ENSURE_REGISTERED` in the runner; add a "Supervised" branch in `ConfigureScheduler`; add a `--modelPath` option if models are loaded at run time during experimentation (the compiled-in model remains the default); add "Supervised" to the `PROTOCOLS` lists in the Python drivers (preferably the shared `common.py`).
6. **4.6 Oracle reference scheduler.** Implement `MpTcpSchedulerOracle`, which uses ground-truth link rates and delays (privileged information) to choose the subflow with the earliest estimated arrival. It is not a competitor; it is an upper-bound reference that shows how much headroom exists and is also the source of the secondary training label.

7 **4.7 Unit and integration tests.** Test eligibility handling (no subflow with window returns false), the fallback path, behaviour with a single subflow, behaviour when a subflow is added late, and a short end-to-end run in the sanity suite comparing the learned scheduler to minRTT.

Deliverables

- Scheduler header, feature extractor with golden test, oracle scheduler, updated runner and drivers, tests.

Exit criteria

`--protocol=Supervised` runs end to end on all presets and dynamic scenarios without crashing; the fallback rate is near zero after start-up; decision overhead is not measurable in simulation wall-clock compared with minRTT; the golden feature test passes.

PHASE 5 — Comparative evaluation (objective ii)

~1.5 weeks
Owner: scheduler/eval

Goal. Answer objective (ii): how does the learned scheduler compare with minRTT, BLEST and ECF on all five metrics?

Experimental matrix

Factor	Levels
Schedulers	Standard-TCP (reference), minRTT, RoundRobin, BLEST, ECF, Supervised, Oracle (upper bound)
Topologies	WiFi + LTE, Ethernet + WiFi, tri-path Ethernet + WiFi + 5G, plus a 2-path high-disparity preset
Conditions	Static; plus each dynamic pattern in the library at default severity
Workloads	Unbounded bulk (30 s) for goodput, throughput, HoL and retransmissions; fixed-size 100 KB / 1 MB / 10 MB for FCT
Repeats	At least 10 seeds per cell (20 for cells where intervals overlap)
Metrics	Goodput, wire throughput, HoL delay (mean, p95), retransmission ratio, FCT (mean, p95), aggregation efficiency, jitter

Tasks

1 **5.1 Select the final model on validation scenarios only.** Compare the shortlisted models by simulated goodput and FCT on a validation scenario set that is disjoint from both training and test. Freeze the choice, and record it in a file committed before the test runs.

2 **5.2 Run the matrix.** Use a driver that parallelizes across seeds, retries failures and writes manifests. Store raw per-run rows and per-100 ms time series.

3 **5.3 Statistics.** Report mean and 95% confidence interval (t-distribution) per cell. For the headline claims, use paired comparisons (same seed and scenario) with a Wilcoxon signed-rank test and an effect size, with a multiple-comparison correction across the scheduler pairs.

4 **5.4 Figures.** Update `plot_benchmark_results.py` to draw bars with error bars; CDFs of FCT and HoL delay; goodput time series with dynamic events marked; per-path traffic share; and a headroom plot showing each scheduler's goodput as a fraction of the oracle's.

5 **5.5 Explain the results.** For each major difference, find the mechanism, for example: the learned scheduler waits for the fast path in the high-disparity case, lowering HoL delay and p95 FCT; or it leaves capacity unused because its delay predictions are conservative. Pair each claim with a diagnostic plot.

Exit criteria

A single results table and a figure set cover all schedulers and metrics with confidence intervals; every headline claim has a significance statement and a mechanism-level explanation.

PHASE 6 — Generalization study (objective iii)

~2 weeks
Owner: all

Goal. Answer the proposal's central question: how well does the supervised scheduler cope with conditions it was not trained on, including dynamic changes in bandwidth, latency and loss? The protocol is described in Section 7.5 and is written down before any test is run.

Tasks

1. **6.1 In-distribution baseline.** Evaluate on held-out scenarios drawn from the same families as training (a new random draw) to establish the no-shift reference performance (G0).
2. **6.2 Parameter extrapolation (G1).** Evaluate beyond the training ranges: delay disparity up to 16x (the existing sweep), rates above the training maximum, loss above 3%, and very small queues.
3. **6.3 Topology shift (G2).** Train on 2-path data and test on the tri-path topology, and (separately) the reverse. This uses the shared per-subflow model design from Phase 3.
4. **6.4 Dynamic shift (G3).** Evaluate under patterns never used for training: for example train with step changes and test with ramps, periodic fading, delay spikes, loss bursts, outages, and the quality swap. This covers the proposal's dynamic bandwidth, latency and loss changes directly.
5. **6.5 Workload shift (G4).** Train on bulk transfers and test on short flows and on/off traffic.
6. **6.6 Generalization gap.** For each shift, compute the relative degradation of every scheduler (metric in shifted condition vs in-distribution, normalized by the oracle). The question is not only whether Supervised degrades but whether it degrades *more* than the heuristics, which are hand-designed to be condition-independent.
7. **6.7 Does diversity help?** Retrain after adding some of the shifted families to the training set and measure how much of the gap closes. This shows whether the limitation is the model class or the training data.
8. **6.8 Ablations.** Permutation importance of each of the seven features; retraining without each feature (does the scheduler need the loss signal? the bandwidth estimate?); noisy-telemetry robustness (add noise to RTT and bandwidth); sensitivity to model size; shared scoring model vs classification formulation; regression labels vs oracle labels.
9. **6.9 Failure analysis.** For the worst shifted cases, plot the decision timeline (chosen subflow, predicted vs actual delay) to show where and why the model fails, for example, extrapolating outside the feature range.

Exit criteria

Every shift family has a result with confidence intervals for all schedulers; the gap analysis answers the central question with evidence either way; the ablations identify which features matter.

PHASE 7 **Write-up and packaging** ~1.5 weeks (starts in Phase 5) Owner: all

Goal. Turn the results into a report that a reader can trust and an examiner can reproduce.

Tasks

1. **7.1 Report structure.** Introduction and objectives; background (MPTCP, schedulers, related ML approaches); simulation methodology (topologies, workloads, metric definitions, measurement windows, statistical method); scheduler designs including the baselines' corrected equations; the learned scheduler (features, labels, model, export); results for objective (ii); the generalization study for objective (iii); discussion; threats to validity; conclusion.
2. **7.2 Threats to validity.** Point-to-point links instead of real Wi-Fi/cellular dynamics; a single simulator (ns-3-mptcp is an older fork); bulk and short-flow workloads rather than real application mixes; a model trained on simulated labels that depend on simulator-specific queue behaviour; and limited topology count. State these plainly. Mention the Phase 0 defects and their correction, which shows rigor rather than weakness.
3. **7.3 Reproducibility package.** A tagged commit, pinned dependencies, the patch files, the scenario files and seeds, the frozen model, a one-command script that regenerates every figure, and the data card.
4. **7.4 Presentation material.** A slide deck and a short demo script (a single command that runs one scenario under minRTT and Supervised and shows the two goodput time series side by side).

7. Key design decisions

7.1 Feature definitions

The seven features named in the proposal are computed per subflow at each scheduling decision. The table specifies the source in ns-3-mptcp and the pitfalls to handle.

Feature	Source	Definition and pitfalls
Smoothed RTT	RTT trace / subflow RTT estimator	Latest smoothed estimate. Undefined before the first ACK; the scheduler probes each subflow once to obtain it.
Minimum RTT	Derived from RTT samples	Minimum over a sliding window (for example 10 s), not the whole run, so that a path change updates it. Needs an explicit "unset" flag (the current 20 ms sentinel is wrong).
Bandwidth estimate	Delivery-rate estimator on `HighestRxAck`	Newly acknowledged bytes per unit time, EWMA over about 100 ms. Underestimates when the sender is application- or window-limited, so use the maximum over a short history, or cwnd/RTT as a floor.
Congestion window	`CongestionWindow` trace	In bytes; also expressed relative to bytes in flight.
Bytes in flight	`BytesInFlight()`	Unacknowledged bytes on the subflow at decision time.
Available window	`AvailableWindow()`	Window headroom; a subflow with less than one MSS is ineligible.
Loss rate	Retransmission counter (Phase 1) or `CongState` transitions	Fraction of segments retransmitted over a sliding window (about 1 s or the last 100 segments). The current cumulative-since-start ratio reacts far too slowly to dynamic loss and must not be used.

Derived features (Phase 3): RTT relative to the fastest subflow, bandwidth share, window headroom as a fraction of cwnd, RTT inflation over own minimum, number of active subflows. These make the model scale-free.

7.2 How to label scheduling decisions

Supervised learning needs a target. For scheduling there is a conceptual difficulty: a run records what happened on the subflow that was *chosen*, not what would have happened on the others (the counterfactual). Simply training a model to copy a heuristic such as ECF produces a scheduler that is at best as good as that heuristic, which does not address the research question. Two labeling strategies avoid this, and the roadmap uses both so they can be compared.

	Strategy A: observed outcome regression	Strategy B: oracle imitation
Target	Measured delivery delay of the segment on the subflow that carried it	The subflow with the earliest true arrival time, computed from ground-truth link parameters and queue state
Handling of the counterfactual	Exploration (epsilon-random policy) makes every subflow appear as the chosen one in many similar states; the model learns delay as a function of (state, subflow) and picks the minimum	Ground truth supplies the best action directly at each decision
Uses privileged information?	No. Features and labels are both observable	Yes at training time only; at run time the model sees only observable features
Strength	Learns from what actually happens, including queueing and HoL effects	Clean label, a true best action for classification
Weakness	Noisy labels (delay depends on other segments); needs exploration data	Oracle model is itself an approximation; may not capture HoL effects
Role	Primary	Secondary / ablation

The comparison between A and B is itself a small research result and is part of the Phase 6 ablations.

7.3 Model class and integration approach

- **Decision-tree-based models, exported to C++.** Trees and small forests are accurate on tabular features, cheap to evaluate (a few dozen comparisons), easy to inspect, and exportable as arrays. This fits the word "lightweight" in the problem statement and the header-only integration pattern already used in the repository.
- **No runtime ML dependency.** Calling Python or ONNX Runtime from inside the ns-3 scheduler would slow every simulation, complicate the build and make runs harder to reproduce. Embedding the model as generated C++ avoids all three.
- **Shared per-subflow scoring.** One model scores each eligible subflow, so it works for any path count and supports topology-shift tests.
- **Offline training only.** The scheduler does not learn during the simulation. This isolates the question of generalization from online-adaptation effects, which can be listed as future work (and relates to the reinforcement learning approach of ReLeS cited in the proposal).

7.4 Metric definitions

Precise definitions matter because the existing metrics were ambiguous (Section 4). All metrics are computed over the same measurement window $[t_0, t_1]$ for every scheduler, where t_1 is the time the sender stops and t_0 excludes the first 10 seconds of slow start for steady-state metrics.

Metric	Definition	Source
Goodput (Mbps)	New application bytes delivered in order to the receiving application in the window divided by the window length. Excludes duplicates and retransmitted data.	PacketSink Rx trace
Wire throughput (Mbps)	All bytes received on the receiver's interfaces, including headers and retransmissions, in the window divided by its length.	Device or FlowMonitor receive counters

Metric	Definition	Source
Aggregation efficiency (%)	Goodput divided by the time-average of the sum of path capacities over the window (use the scripted capacity timeline under dynamics).	Scenario file and goodput
HoL delay (ms)	For each segment: time of in-order release to the application minus time of arrival at the meta socket. Report mean and 95th percentile. Also mean out-of-order buffer occupancy (bytes).	Meta-socket hook (Phase 1)
Retransmission ratio (%)	Retransmitted segments divided by all segments sent, per subflow and overall.	Subflow counter (Phase 1)
Flow completion time (s)	Time from the first byte handed to the sender until the sink has received all bytes of a fixed-size flow. Mean and 95th percentile over repeats.	Sink trace with MaxBytes flows
Jitter (ms)	Standard deviation of inter-delivery times of in-order segments at the sink.	Sink trace
Decision overhead	Microseconds per decision (microbenchmark) and the fallback rate (fraction of decisions that used the safety rule).	Microbenchmark, scheduler counter
Oracle-normalized score	Scheduler metric divided by the Oracle scheduler's metric on the same scenario and seed.	Derived
Generalization gap	Oracle-normalized score in-distribution minus the score under shift; compared across schedulers.	Derived

7.5 Evaluation and generalization protocol

The scenario families below are fixed before any test result is inspected. Training uses families F0 and F1 only. Every other family is a distribution shift in a named dimension, so a failure can be attributed to a cause.

Family	Description	Training?	Test label
F0	Static 2-path scenarios sampled by Latin hypercube over rate 2-50 Mbps, delay 2-100 ms, loss 0-3%, queue 0.5-3 x BDP	Yes (train split)	G0: new draw from the same family
F1	F0 plus mild dynamics: bandwidth or loss step changes only	Yes (train split)	G0
F2	Extrapolated static: delay disparity up to 16x, rates above 50 Mbps, loss up to 5-8%, very small queues	No	G1: parameter shift
F3	Three-path topology (Ethernet + Wi-Fi + 5G preset and variants)	No	G2: topology shift
F4	Unseen dynamics: ramps, periodic fading, delay spikes, loss bursts, path outage, quality swap, handover	No	G3: dynamic shift (bandwidth, latency, loss)
F5	Unseen workloads: short flows (100 KB, 1 MB), on/off traffic	No	G4: workload shift

Statistical method

- At least 10 independent seeds per cell, with scenario parameters held fixed across schedulers so comparisons are paired.
- Mean with 95% confidence interval (t-distribution). Wilcoxon signed-rank test for paired scheduler comparisons, with Holm correction across pairs, and an effect size (for example median relative difference).
- Report negative and null results with the same prominence as positive ones.
- Never tune on the test families. Hyperparameters and the final model are fixed on the validation split.

8. Target repository layout

The layout below keeps the team's existing directory names, so nothing needs to be moved, and adds the directories required by the roadmap. Items marked *new* do not exist yet.

```
mptcp_scheduler/
  README.md                          (new)  build, run, reproduce
  requirements.txt                   (new)  pinned Python dependencies
  patches/                           (new)  ns-3-mptcp changes kept under version control
    0001-subflow-retx-trace.diff
    0002-meta-rx-hol-trace.diff
    apply_patches.sh
  scripts/
    run_all.sh                       (new)  build + all experiments + all figures
  classical_schedulers/
    telemetry_tracker.h                     (fixed: bandwidth estimator, loss window, sentinel)
    telemetry_scheduler.h            (new)  shared base class: lazy registration + refresh
    mptcp-scheduler-min-rtt.h               (uses base class)
    mptcp-scheduler-round-robin.h
    mptcp-scheduler-blest.h                 (re-implemented from the paper)
    mptcp-scheduler-ecf.h                   (re-implemented from the paper)
    mptcp-scheduler-oracle.h         (new)  upper-bound reference using ground truth
  adaptive_supervised_scheduler/
    feature_extractor.h              (new)  ONE feature function shared by logger and scheduler
    decision_logger.h                (new)
    model_generated.h                (new)  produced by ml/export_cpp.py
    mptcp-scheduler-supervised.h     (new)
    tests/                           (new)  golden-feature test, fallback tests
  experiments/
    run_modular_experiment.cc               (generic N-path, --seed --run --events --logDecisions)
    topologies/
      link_profile.h
      dynamic_controller.h                  (delay/rate/loss events, events-file loader)
      modular_mptcp_topology.h              (single N-path builder)
    metrics/metrics_collector.h      (new)  goodput, wire throughput, HoL, retx, FCT, time series
    sanity/                          (new)  expected-outcome tests for the baseline
    common.py                        (new)  shared driver code
    run_benchmarks.py / run_disparity_sweep.py / run_dynamic_handover.py
    run_generalization.py            (new)
    plot_benchmark_results.py               (CIs, CDFs, time series)
  dataset/
    raw/                                    decision logs and per-run outcomes
    processed/                              labeled data, splits.json, DATA_CARD.md
  ml/
    sample_scenarios.py              (new)
    build_dataset.py                 (new)
    train.py, evaluate_offline.py    (new)
    export_cpp.py, test_export.py    (new)
    models/                                 frozen model + MODEL_CARD.md
  results/
    data/  plots/  timeseries/  manifests/
  docs/
    report/ slides/                  (new)
```

9. Team split and schedule

9.1 Suggested division of work

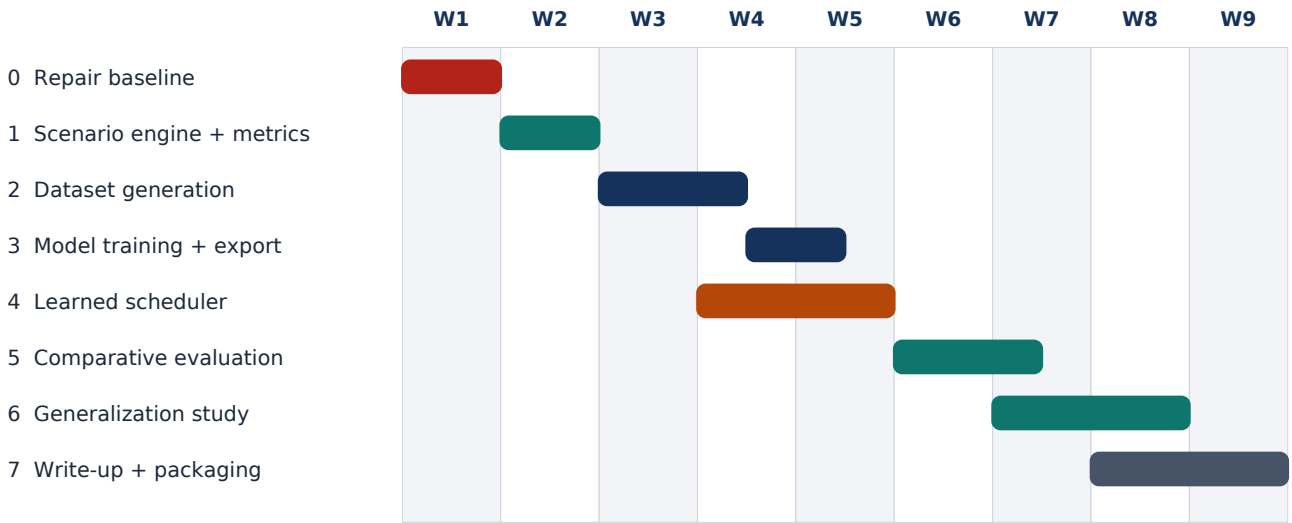
The three roles below reduce hand-offs and let work proceed in parallel after the interfaces are agreed in Phase 0. Roles can rotate; what matters is that each interface has one named owner.

Role	Primary phases	Owns (interfaces)	Hands off to
A. Simulator and metrics	0, 1, then supports 5	Topology builder, dynamics, metrics collector, ns-3 patches, run manifests, sanity suite	B: decision-log schema and outcome files; C: the metrics CSV schema
B. Dataset and ML	2, 3, then 6 (retraining, ablations)	Scenario sampler, dataset and splits, training, model export and equivalence test	C: the generated model header and the feature definition
C. Scheduler and evaluation	0.3, 4, 5, 6	Repaired BLEST/ECF, feature extractor, supervised scheduler, oracle, experiment matrix, statistics and figures	A, B: requirements and bug reports; all: the figure set

Interface agreements to settle in the first week: the decision-log column schema (Appendix A.3), the metrics CSV schema, the events-file format, the seven-feature definitions (Section 7.1), and the shared feature function's signature.

9.2 Timeline

Effort is estimated at about nine weeks for three people. Phases 3 and 4 overlap, since the scheduler plumbing can be built and tested with a hand-written stand-in model while data is being collected. The write-up starts in week 7 with the methodology sections, which do not depend on final results. Scale the plan to the actual deadline by shortening Phases 5 and 6 first (fewer seeds, fewer pattern families), never by skipping Phase 0.



Milestones

Milestone	Week	Evidence
M1: Trustworthy baseline	1	Sanity suite passes; BLEST/ECF/minRTT differ; metrics corrected; repository clean and committed
M2: Full measurement stack	2	Dynamics and all five metrics validated on scripted scenarios
M3: Dataset frozen	3.5	Data card, splits and hash recorded
M4: Model frozen	5	Model chosen on validation scenarios and committed before test runs
M5: Objective (ii) results	6.5	Comparison tables with confidence intervals and figures
M6: Objective (iii) results	8	Gap analysis, ablations, failure analysis
M7: Submission package	9	Report, slides, reproducibility package

10. Risk register

Risk	Likelihood	Impact	Mitigation
Fixing the telemetry reveals deeper problems in the ns-3-mptcp fork (for example MPTCP still underperforms single-path TCP)	Medium	High	Time-box investigation in Phase 0; test congestion control and buffer settings; if unresolved, document the limitation and evaluate relative to the repaired baselines.
The learned scheduler does not beat the heuristics	Medium	Medium	This is a valid outcome. Report where and why, using the oracle headroom and failure analysis. Check that the model is not being held back by noisy features or too little exploration data.
Train/serve skew: logged features differ from run-time features	Medium	High	One shared feature function; golden-file test comparing C++ and Python vectors; check the fallback rate.
Label noise from interacting segments	High	Medium	Scenario-grouped validation; queue-state features; compare regression and oracle labels.
Leakage between train and test through row-wise or scenario-correlated splits	Medium	High	Split by scenario and by family; freeze splits before training; review the manifests.
Simulation time too long for the planned matrix	Medium	Medium	Measure time per run in Phase 0; parallelize over cores; reduce seeds for exploratory runs and keep 10 for headline cells.
ns-3 patches break or get lost	Low	High	Keep patches under `patches/` with an apply script; record the ns-3 commit in manifests.
Exploration data degrades scheduler runs and is mistakenly reported as results	Low	Medium	Separate output directories and manifest flags; plotting scripts read only the evaluation directory.
Point-to-point links do not capture Wi-Fi/cellular behaviour	High	Medium	Use the dynamics library to emulate variability; list as a threat to validity; optionally validate one scenario against a trace-driven rate profile.
Scope creep (deep learning, online learning, new protocols)	Medium	Medium	Treat as future work; the proposal asks for a lightweight supervised scheduler and a generalization study.

Appendix A. Reference snippets

These snippets are starting points that show the intended shape of each interface. They have not been compiled against the repository, so treat them as design sketches and adapt names and includes as needed.

A.1 Shared base class with lazy telemetry registration (Phase 0, task 0.1)

```
// classical_schedulers/telemetry_scheduler.h
class TelemetryScheduler : public MpTcpScheduler
{
public:
  TelemetryScheduler () : m_meta (0), m_registered (0) {}
  virtual void SetMeta (Ptr<MpTcpSocketBase> meta) { m_meta = meta; m_registered = 0; }

protected:
  // Subflows only exist after the handshake, so register lazily on first use.
  void EnsureRegistered ()
  {
    int n = m_meta->GetNActiveSubflows ();
    for (int i = m_registered; i < n; ++i)
      {
        // connects the RTT, cwnd and ack traces
        TelemetryTracker::Get ().RegisterSubflow (i, m_meta->GetSubflow (i));
      }
    m_registered = n;
  }

  // Call at the start of every GenerateMapping().
  void RefreshAll ()
  {
    EnsureRegistered ();
    for (int i = 0; i < m_registered; ++i)
      {
        TelemetryTracker::Get ().UpdateFromSocket (i, m_meta->GetSubflow (i));
      }
  }

  Ptr<MpTcpSocketBase> m_meta;
  int m_registered;
};
```

A.2 Events file and planned command lines

Flags marked with an asterisk do not exist yet and are added in Phases 0, 1 and 2.

```
# events/step_down.csv   (blank cell = unchanged)
time_s,path,rate_mbps,delay_ms,loss
10.0,0,6,,
20.0,0,12,,
15.0,1,,80,
18.0,1,,,0.05

# one evaluation run
./waf --run "scratch/run_experiment --topology=WiFi_Cellular --protocol=Supervised \
    --duration=30 --seed=1 --run=3 --events=events/step_down.csv"        # seed*, run*, events*

# one data-collection run
./waf --run "scratch/run_experiment --topology=TriPath --protocol=EpsRandom --epsilon=0.3 \
    --duration=20 --seed=7 --run=1 --events=events/s0412.csv \
    --logDecisions=dataset/raw/s0412_eps03.csv"   # EpsRandom*, epsilon*, logDecisions*
```

A.3 Decision-log schema (Phase 2)

One row per successful scheduling decision. Slots k = 0 to 3 are padded; `valid_k` is 0 for unused slots.

Column group	Columns	Notes
Identity	`run_id, scenario_id, policy, t_s, dsn, len, chosen, n_sf`	dsn minus initial dsn gives the stream offset used for the outcome join
Per-slot features (k = 0..3)	`valid_k, rtt_ms_k, minrtt_ms_k, bw_mbps_k, cwnd_B_k, inflight_B_k, avail_B_k, loss_k`	The seven features of the proposal, plus validity
Joined outcome (after build_dataset)	`send_t_s, deliver_t_s, delay_ms, hol_ms`	hol_ms present if the Phase 1 hook exists
Oracle label (secondary)	`oracle_best, oracle_ect_ms_k`	From ground-truth link state, training only

A.4 Generated model header (Phase 3, task 3.6)

```
// adaptive_supervised_scheduler/model_generated.h  (generated by ml/export_cpp.py; do not edit)
namespace ns3 { namespace supervised_model {

static const int   kNumFeatures = 12;
static const int   kNumTrees    = 50;
static const float kMean[kNumFeatures]  = { /* ... */ };
static const float kScale[kNumFeatures] = { /* ... */ };

// Flat node arrays across all trees. kFeat[n] < 0 marks a leaf.
static const short kFeat[]  = { /* ... */ };
static const float kThr[]   = { /* ... */ };
static const short kLeft[]  = { /* ... */ };
static const short kRight[] = { /* ... */ };
static const float kVal[]   = { /* ... */ };
static const short kRoot[kNumTrees] = { /* ... */ };

// Predicted delivery delay (ms) if the next segment is sent on the scored subflow.
inline float Predict (const float* x /* already scaled */)
{
  float s = 0.0f;
  for (int t = 0; t < kNumTrees; ++t)
    {
      int n = kRoot[t];
      while (kFeat[n] >= 0)
        n = (x[kFeat[n]] <= kThr[n]) ? kLeft[n] : kRight[n];
      s += kVal[n];
    }
  return s / kNumTrees;
}
}}
```

A.5 Decision procedure of the learned scheduler (Phase 4, task 4.3)

```
bool GenerateMapping (int& id, SequenceNumber64& dsn, uint16_t& length)
{
  if (metaWindow == 0 || nothingToSend) return false;
  RefreshAll ();                                  // telemetry for every subflow

  best = -1; bestDelay = +inf;
  for each subflow i with AvailableWindow(i) >= MSS(i):
      x = FeatureExtractor::Build (telemetry, i, allSubflows);   // same code as the logger
      // invalid features: use the repaired minRTT rule and count the fallback
      if (!AllFinite (x)) { return Fallback (id, dsn, length); }
      d = supervised_model::Predict (Scale (x));
      if (d < bestDelay) { best = i; bestDelay = d; }

  if (best < 0) return false;                     // no eligible subflow: wait

  // optional wait rule: hold back if a faster path is predicted to free up soon enough
  if (UseWaitRule && ShouldWaitForFaster (best, bestDelay)) return false;

  id = best;
  dsn = nextTxSeq;
  length = min (MSS, subflowWindow(best), metaWindow);
  logger.Log (now, dsn, length, best, telemetry);   // only when --logDecisions is set
  return length > 0;
}
```

Appendix B. Baseline results as recorded (provisional)

These are the numbers currently in `results/data/`. They are reproduced here as the starting point and should be replaced after Phase 0. See Section 4 for why they cannot be relied on.

B.1 Four scenarios, 8 s runs (multitopology_benchmark.csv): throughput, mean delay and path-0 share

Scenario	Scheduler	Throughput (Mbps)	Mean delay (ms)	Path 0 share (%)
WiFi + LTE (18)	Standard-TCP	4.95	17.70	100.0
	FastestRTT	4.59	26.07	81.8
	RoundRobin	3.64	34.29	60.7
	BLEST	4.59	26.07	81.8
	ECF	3.62	30.72	71.4
Ethernet + WiFi (40)	Standard-TCP	29.28	37.27	100.0
	FastestRTT	24.27	16.23	98.7
	RoundRobin	12.36	14.46	57.4
	BLEST	24.27	16.23	98.7
	ECF	12.53	15.15	64.1
Tri-path (55)	Standard-TCP	37.34	216.66	100.0
	FastestRTT	26.87	12.95	99.0
	RoundRobin	4.82	27.29	36.8
	BLEST	26.87	12.95	99.0
	ECF	6.24	20.16	56.0
WiFi + LTE, fading 4-7 s (18)	Standard-TCP	5.24	18.41	100.0
	FastestRTT	3.41	26.98	78.2
	RoundRobin	3.01	34.91	57.0
	BLEST	3.41	26.98	78.2
	ECF	3.03	30.68	71.1

Observations: the FastestRTT and BLEST rows are identical in every scenario; ECF and RoundRobin behave similarly; the tri-path single-path throughput (37.34 Mbps) exceeds the 30 Mbps link; and path-0 share under FastestRTT is 98.7 to 99% on Ethernet-based topologies, consistent with a primary-first spill-over rule rather than an RTT-aware one.

B.2 Delay-disparity sweep, session goodput in Mbps (disparity_sweep.csv)

Disparity label	Standard-TCP	FastestRTT	RoundRobin	BLEST	ECF	Actual path-1 / path-0 delay ratio
1x	5.39	4.51	3.69	4.51	4.00	3.0 (default 15 ms / 45 ms)
2x	5.39	6.36	4.93	6.36	4.42	2.0
4x	5.39	4.61	2.97	4.61	3.62	4.0
8x	5.39	3.26	1.29	3.26	1.95	8.0
16x	5.39	2.12	1.14	2.12	1.67	16.0

The 1x row actually has a 3x delay ratio (defect 6), which explains the non-monotonic curve. Single-path TCP is constant because it uses path 0 only.

Appendix C. References

[1] A. Ford, C. Raiciu, M. Handley, O. Bonaventure and C. Paasch, "TCP extensions for multipath operation with multiple addresses," RFC 8684, IETF, Mar. 2020.

[2] C. Paasch, S. Ferlin, O. Alay and O. Bonaventure, "Experimental evaluation of multipath TCP schedulers," ACM SIGCOMM Workshop on Capacity Sharing (CSWS), 2014, pp. 27-32.

[3] Y.-s. Lim, E. M. Nahum, D. Towsley and R. J. Gibbens, "ECF: An MPTCP path scheduler to manage heterogeneous paths," ACM CoNEXT, 2017, pp. 147-159.

[4] S. Ferlin, Ö. Alay, O. Mehani and R. Boreli, "BLEST: Blocking estimation-based MPTCP scheduler for heterogeneous networks," IFIP Networking, 2016, pp. 431-439.

[5] H. Zhang, W. Li, S. Gao, X. Wang and B. Ye, "ReLeS: A neural adaptive multipath scheduler based on deep reinforcement learning," IEEE INFOCOM, 2019, pp. 1648-1656.

The BLEST and ECF equations referenced in Phase 0 task 0.3 should be taken from [4] and [3] directly while implementing; the description in this document is a summary and is not a substitute for the papers.